

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА» (СПбГУТ)
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ПРОГРАММНОЙ
ИНЖЕНЕРИИ (ИТПИ)
КАФЕДРА ПРОГРАММНОЙ ИНЖЕНЕРИИ И ВЫЧИСЛИТЕЛЬНОЙ
ТЕХНИКИ (ПИиВТ)

Дисциплина: «Разработка приложений искусственного интеллекта в
киберфизических системах»

Лабораторная работа №2.

Тема: «Разработка экспертных систем, базирующихся на правилах»

Выполнил:
студент группы ИКПИ-32
Филимонов Д.Е.

Принял:
Березкин А. А.
Подпись _____

Санкт-Петербург
2026 г.

Цель работы

В ходе лабораторной работы необходимо разработать экспертную систему, основанную на правилах, для диагностики инфекционных заболеваний человека по совокупности симптомов с учётом того, что каждый симптом может указывать на несколько болезней с разной степенью уверенности. В процессе выполнения работы необходимо реализовать базу знаний, содержащую не менее 30 продукционных правил, механизм логического вывода, рабочую память и пользовательский интерфейс, обеспечивающий диалог с системой. Также следует предусмотреть взаимодействие с экспертной системой различных типов пользователей, включая неспециалистов, специалистов, студентов, экспертов и инженеров по знаниям, каждый из которых имеет свои задачи при работе с системой.

Ход работы

1. Анализ предметной области

На первом этапе были изучены инфекционные заболевания человека и их характерные симптомы. В результате анализа были выделены 15 основных заболеваний: грипп, COVID-19, ангина, бронхит, пневмония, кишечная инфекция, ветрянка, корь, краснуха, менингит, гепатит А, острая ВИЧ-инфекция, герпес, генитальный герпес и мононуклеоз. Для каждого заболевания определены ключевые симптомы, такие как высокая температура, кашель, боль в горле, потеря обоняния, сыпь, диарея, тошнота и другие.

2. Проектирование базы знаний

База знаний спроектирована в виде набора продукционных правил вида «ЕСЛИ (симптомы), ТО (заболевание)». Каждому правилу присвоен коэффициент уверенности от 0 до 1, отражающий диагностическую значимость данной комбинации симптомов. Всего было разработано 33 правила.

Примеры правил:

- Правило R4: ЕСЛИ высокая_температура, сухой_кашель, потеря_обоняния, одышка ТО COVID-19 (уверенность 0,95)
- Правило R1: ЕСЛИ высокая_температура, озноб, головная_боль, ломота_в_теле ТО грипп (уверенность 0,9)
- Правило R16: ЕСЛИ тошнота, рвота, диарея, температура ТО кишечная_инфекция (уверенность 0,9)

```

15 # База знаний
16 class KnowledgeBase:
17     def __init__(self):
18         self.rules: List[Rule] = []
19         self._init_rules()
20
21     def _init_rules(self):
22         # Инфекционные заболевания и их симптомы (с разной степенью уверенности)
23         # Грипп
24         self.rules.append(Rule("R1", ["высокая температура", "озноб", "головная боль", "ломота в теле"], "грипп", 0.9))
25         self.rules.append(Rule("R2", ["высокая температура", "кашель", "боль в горле"], "грипп", 0.7))
26         self.rules.append(Rule("R3", ["температура", "слабость", "насморк"], "грипп", 0.6))
27
28         # COVID-19
29         self.rules.append(Rule("R4", ["высокая температура", "сухой кашель", "потеря обоняния", "одышка"], "covid19", 0.95))
30         self.rules.append(Rule("R5", ["потеря вкуса", "потеря обоняния", "усталость"], "covid19", 0.85))
31         self.rules.append(Rule("R6", ["температура", "сухой кашель", "боль в груди"], "covid19", 0.75))
32
33         # Ангина (стрептококковая инфекция)
34         self.rules.append(Rule("R7", ["высокая температура", "сильная боль в горле", "увеличенные миндалины"], "ангина", 0.9))
35         self.rules.append(Rule("R8", ["боль в горле", "гнойные пробки", "температура"], "ангина", 0.85))
36         self.rules.append(Rule("R9", ["боль при глотании", "увеличенные лимфоузлы", "температура"], "ангина", 0.7))
37
38         # Бронхит
39         self.rules.append(Rule("R10", ["влажный кашель", "температура", "слабость"], "бронхит", 0.8))
40         self.rules.append(Rule("R11", ["кашель с мокротой", "хрипы в груди", "одышка"], "бронхит", 0.85))

```

Рисунок 1 - Фрагмент программного кода, содержащий реализацию базы знаний с 33 правилами.

3. Реализация механизма логического вывода

Механизм логического вывода реализован на основе прямого цепочки рассуждений (forward chaining). Алгоритм работает следующим образом: система собирает от пользователя факты о наличии симптомов, после чего последовательно перебирает все правила. Если все условия правила выполняются, правило активируется, а соответствующее заболевание заносится в рабочую память.

Уверенность диагноза вычисляется по формуле:

$$\text{Итоговая_уверенность} = \text{Уверенность_пользователя_в_симптоме} \times \text{Уверенность_правила}$$

Если одно заболевание выводится несколькими правилами, итоговая уверенность принимается как максимальное значение среди всех сработавших правил.

```

def forward_chaining(self, explain_mode: bool = False) -> Dict[str, float]:
    changed = True
    while changed:
        changed = False
        for rule in self.kb.get_rules():
            # Проверяем, не выведено ли уже это заключение
            if rule.conclusion in self.derived_diseases:
                continue

            # Проверяем, все ли условия выполняются
            rule_confidence = 1.0
            all_conditions_met = True

            for cond in rule.conditions:
                if cond in self.facts:
                    rule_confidence *= self.facts[cond]
                else:
                    all_conditions_met = False
                    break

            if all_conditions_met:
                # Уверенность правила умножается на уверенность условий
                final_conf = rule_confidence * rule.confidence
                if rule.conclusion in self.derived_diseases:
                    self.derived_diseases[rule.conclusion] = max(
                        self.derived_diseases[rule.conclusion], final_conf
                    )
                else:
                    self.derived_diseases[rule.conclusion] = final_conf

                changed = True

            if explain_mode:
                self.explanation.append(
                    f"Активировано правило {rule.name}: {' '.join(rule.conditions)} -> {rule.conclusion} (уверенность: {final_conf:.2f})"
                )

```

Рисунок 2 - Реализация класса InferenceEngine, отвечающего за логический вывод.

4. Разработка рабочей памяти

Рабочая память реализована в виде двух словарей (ассоциативных массивов). Первый словарь facts хранит введенные пользователем симптомы и их уверенность. Второй словарь derived_diseases хранит выведенные системой заболевания с итоговыми коэффициентами уверенности. Перед каждым новым сеансом диагностики рабочая память очищается, что позволяет проводить неограниченное количество консультаций.

```

100     def __init__(self, knowledge_base: KnowledgeBase):
101         self.kb = knowledge_base
102         self.facts: Dict[str, float] = {} # симптом -> уверенность (0..1)
103         self.derived_diseases: Dict[str, float] = {} # болезнь -> уверенность
104         self.explanation: List[str] = [] # для режима студента
105
106     def add_fact(self, symptom: str, confidence: float = 1.0):
107         self.facts[symptom] = confidence
108
109     def reset(self):
110         self.facts = {}
111         self.derived_diseases = {}
112         self.explanation = []

```

Рисунок 3 - Структура данных для хранения фактов и выведенных заключений.

5. Создание пользовательского интерфейса

Пользовательский интерфейс реализован в виде консольного диалога на ограниченном естественном языке. При запуске системы пользователю предлагается выбрать тип пользователя, после чего отображается главное меню с доступными действиями.

```
=====
ЭКСПЕРТНАЯ СИСТЕМА: Диагностика инфекционных заболеваний
=====

--- Выберите тип пользователя ---
1. Обычный пользователь (получить диагноз)
2. Студент (диагностика с объяснением)
3. Эксперт (можно добавлять правила)
4. Инженер по знаниям (полный доступ к БЗ)
Ваш выбор: 1
Режим: Обычный пользователь

--- Главное меню ---
1. Пройти диагностику
2. Сменить тип пользователя
0. Выход

Выберите действие: █
```

Рисунок 4 – Главное меню программы.

В режиме диагностики система последовательно задаёт вопросы о наличии симптомов. Пользователь отвечает «да» или «нет». Для экспертов и инженеров по знаниям предусмотрена возможность указания числовой уверенности в симптоме (от 0 до 1).

```
--- Диагностика инфекционных заболеваний ---
Отвечайте 'да' или 'нет' на вопросы о симптомах.

Есть ли у вас симптом: боль_в_горле? (да/нет): да
Есть ли у вас симптом: боль_в_груди? (да/нет): да
Есть ли у вас симптом: боль_в_животе? (да/нет): да
Есть ли у вас симптом: боль_в_правом_подреберье? (да/нет): да
Есть ли у вас симптом: боль_в_суставах? (да/нет): да
Есть ли у вас симптом: боль_при_глотании? (да/нет): да
Есть ли у вас симптом: влажный_кашель? (да/нет): да
Есть ли у вас симптом: высокая_температура? (да/нет): да
Есть ли у вас симптом: гнойные_пробки? (да/нет): да
Есть ли у вас симптом: головная_боль? (да/нет): да
Есть ли у вас симптом: диарея? (да/нет): да
```

Рисунок 5 – Процесс диагностики.

На рисунке показан диалог с пользователем, где система задаёт вопросы о наличии симптомов.

6. Реализация режимов для разных типов пользователей

В соответствии с заданием были реализованы четыре режима работы для пяти типов пользователей:

Режим «Обычный пользователь» - система просто задаёт вопросы и выводит диагноз без лишней информации.

Режим «Студент» - дополнительно отображается пошаговое объяснение хода рассуждений: какие правила активировались и с какой уверенностью.

```

--- Результаты диагностики ---

--- Ход рассуждений ---
• Активировано правило R1: высокая_температура, озноб, головная_боль, ломота_в_теле → грипп (уверенность: 0.90)
• Активировано правило R4: высокая_температура, сухой_кашель, потеря_обоняния, одышка → covid19 (уверенность: 0.95)
• Активировано правило R7: высокая_температура, сильная_боль_в_горле, увеличенные_миндалины → ангина (уверенность: 0.90)
• Активировано правило R10: влажный_кашель, температура, слабость → бронхит (уверенность: 0.80)
• Активировано правило R13: высокая_температура, кашель_с_кровью, одышка, боль_в_груди → пневмония (уверенность: 0.95)
• Активировано правило R16: тошнота, рвота, диарея, температура → кишечная_инфекция (уверенность: 0.90)
• Активировано правило R19: температура, сыпь_пузырьковая, зуд → ветрянка (уверенность: 0.95)
• Активировано правило R21: высокая_температура, кашель, сыпь, пятна_бельского → корь (уверенность: 0.95)
• Активировано правило R23: температура, увеличенные_лимфоузлы, розовая_сыпь → краснуха (уверенность: 0.90)
• Активировано правило R25: сильная_головная_боль, ригидность_затылочных_мышц, высокая_температура → менингит (уверенность: 0.95)
• Активировано правило R27: желтуха, температура, тошнота, темная_моча → гепатит_А (уверенность: 0.90)
• Активировано правило R29: температура, увеличенные_лимфоузлы, сыпь, боль_в_горле → вич_острая (уверенность: 0.70)
• Активировано правило R31: пузырьки_на_губах, жжение, зуд → герпес (уверенность: 0.95)
• Активировано правило R32: пузырьки_на_гениталиях, температура, увеличенные_лимфоузлы → генитальный_герпес (уверенность: 0.90)
• Активировано правило R33: высокая_температура, сильная_боль_в_горле, увеличенные_лимфоузлы, усталость → мононуклеоз (уверенность: 0.85)

```

Рисунок 6 – Режим студента с объяснением.

На рисунке показан результат диагностики в режиме студента с отображением хода рассуждений.

Режим «Эксперт» - пользователь может добавлять новые правила в базу знаний без остановки системы.

```

--- Добавление нового правила ---
Имя правила (например, R34): R34
Введите симптомы (условия) через запятую (например: температура, кашель, боль):
Симптомы: температура, кашель
Заключение (название болезни): болячка
Уверенность правила (0-1): 1
Правило R34 успешно добавлено!

```

Рисунок 7 – Добавление нового правила.

Режим «Инженер по знаниям» - доступно добавление, удаление и просмотр всех правил базы знаний.

```

--- Все правила базы знаний ---
1. R1: ЕСЛИ высокая_температура, озноб, головная_боль, ломота_в_теле ТО грипп (уверенность: 0.9)
2. R2: ЕСЛИ высокая_температура, кашель, боль_в_горле ТО грипп (уверенность: 0.7)
3. R3: ЕСЛИ температура, слабость, насморк ТО грипп (уверенность: 0.6)
4. R4: ЕСЛИ высокая_температура, сухой_кашель, потеря_обоняния, одышка ТО covid19 (уверенность: 0.95)
5. R5: ЕСЛИ потеря_вкуса, потеря_обоняния, усталость ТО covid19 (уверенность: 0.85)
6. R6: ЕСЛИ температура, сухой_кашель, боль_в_груди ТО covid19 (уверенность: 0.75)
7. R7: ЕСЛИ высокая_температура, сильная_боль_в_горле, увеличенные_миндалины ТО ангина (уверенность: 0.9)
8. R8: ЕСЛИ боль_в_горле, гнойные_пробки, температура ТО ангина (уверенность: 0.85)
9. R9: ЕСЛИ боль_при_глотании, увеличенные_лимфоузлы, температура ТО ангина (уверенность: 0.7)
10. R10: ЕСЛИ влажный_кашель, температура, слабость ТО бронхит (уверенность: 0.8)
11. R11: ЕСЛИ кашель_с_мокротой, хрипы_в_груди, одышка ТО бронхит (уверенность: 0.85)
12. R12: ЕСЛИ сухой_кашель, боль_в_груди, температура ТО бронхит (уверенность: 0.7)
13. R13: ЕСЛИ высокая_температура, кашель_с_кровью, одышка, боль_в_груди ТО пневмония (уверенность: 0.95)
14. R14: ЕСЛИ озноб, сильный_кашель, слабость, потливость ТО пневмония (уверенность: 0.85)
15. R15: ЕСЛИ температура_выше_39, затрудненное_дыхание, цианоз ТО пневмония (уверенность: 0.9)
16. R16: ЕСЛИ тошнота, рвота, диарея, температура ТО кишечная_инфекция (уверенность: 0.9)
17. R17: ЕСЛИ боль_в_животе, диарея, слабость ТО кишечная_инфекция (уверенность: 0.8)
18. R18: ЕСЛИ рвота, температура, обезвоживание ТО кишечная_инфекция (уверенность: 0.75)
19. R19: ЕСЛИ температура, сыпь_пузырьковая, зуд ТО ветрянка (уверенность: 0.95)
20. R20: ЕСЛИ сыпь_на_всем_теле, температура, слабость ТО ветрянка (уверенность: 0.85)
21. R21: ЕСЛИ высокая_температура, кашель, сыпь, пятна_бельского ТО корь (уверенность: 0.95)
22. R22: ЕСЛИ температура, сыпь, светобоязнь ТО корь (уверенность: 0.85)
23. R23: ЕСЛИ температура, увеличенные_лимфоузлы, розовая_сыпь ТО краснуха (уверенность: 0.9)
24. R24: ЕСЛИ сыпь, боль_в_суставах, температура ТО краснуха (уверенность: 0.75)
25. R25: ЕСЛИ сильная_головная_боль, ригидность_затылочных_мышц, высокая_температура ТО менингит (уверенность: 0.95)
26. R26: ЕСЛИ тошнота, светобоязнь, спутанность_сознания, температура ТО менингит (уверенность: 0.9)
27. R27: ЕСЛИ желтуха, температура, тошнота, темная_моча ТО гепатит_А (уверенность: 0.9)
28. R28: ЕСЛИ боль_в_правом_подреберье, слабость, потеря_аппетита ТО гепатит_А (уверенность: 0.8)
29. R29: ЕСЛИ температура, увеличенные_лимфоузлы, сыпь, боль_в_горле ТО вич_острая (уверенность: 0.7)
30. R30: ЕСЛИ ночная_потливость, потеря_веса, длительная_лихорадка ТО вич_острая (уверенность: 0.75)
15. R15: ЕСЛИ температура_выше_39, затрудненное_дыхание, цианоз ТО пневмония (уверенность: 0.9)
16. R16: ЕСЛИ тошнота, рвота, диарея, температура ТО кишечная_инфекция (уверенность: 0.9)

```

Рисунок 8 – Список правил.

На рисунке показан список правил базы знаний в отформатированном виде.

7. Тестирование системы

Тестирование проводилось на нескольких наборах симптомов. Ниже приведены результаты некоторых тестов.

Тест 1: Симптомы, характерные для COVID-19

- Входные данные: высокая температура, сухой кашель, потеря обоняния, одышка
- Результат: COVID-19 с уверенностью 0,95

```
Есть ли у вас симптом: сильный_кашель? (да/нет): да
Есть ли у вас симптом: слабость? (да/нет): нет
Есть ли у вас симптом: спутанность_сознания? (да/нет): нет
Есть ли у вас симптом: сухой_кашель? (да/нет): да
Есть ли у вас симптом: сыпь? (да/нет): нет
Есть ли у вас симптом: сыпь_на_всем_теле? (да/нет): нет
Есть ли у вас симптом: сыпь_пузырьковая? (да/нет): нет
Есть ли у вас симптом: темная_моча? (да/нет): нет
Есть ли у вас симптом: температура? (да/нет): нет
Есть ли у вас симптом: температура_выше_39? (да/нет): нет
Есть ли у вас симптом: тошнота? (да/нет): нет
Есть ли у вас симптом: увеличенные_лимфоузлы? (да/нет): нет
Есть ли у вас симптом: увеличенные_миндалины? (да/нет): нет
Есть ли у вас симптом: усталость? (да/нет): нет
Есть ли у вас симптом: хрипы_в_груди? (да/нет): нет
Есть ли у вас симптом: цианоз? (да/нет): нет

--- Результаты диагностики ---

--- Возможные диагнозы (по убыванию уверенности) ---
1. covid19: уверенность 0.95 (95%)

--- Рекомендации ---
С высокой вероятностью у вас covid19. Рекомендуется обратиться к врачу.
```

Рисунок 9 – Тест 1.

Тест 2: Симптомы, характерные для кишечной инфекции

- Входные данные: тошнота, рвота, диарея, температура
- Результат: кишечная инфекция с уверенностью 0,9

```

Есть ли у вас симптом: темная_моча? (да/нет): нет
Есть ли у вас симптом: температура? (да/нет): да
Есть ли у вас симптом: температура_выше_39? (да/нет): нет
Есть ли у вас симптом: тошнота? (да/нет): да
Есть ли у вас симптом: увеличенные_лимфоузлы? (да/нет): нет
Есть ли у вас симптом: увеличенные_миндалины? (да/нет): нет
Есть ли у вас симптом: усталость? (да/нет): нет
Есть ли у вас симптом: хрипы_в_груди? (да/нет): нет
Есть ли у вас симптом: цианоз? (да/нет): нет

--- Результаты диагностики ---

--- Возможные диагнозы (по убыванию уверенности) ---
1. кишечная_инфекция: уверенность 0.90 (90%)

--- Рекомендации ---
С высокой вероятностью у вас кишечная_инфекция. Рекомендуется обратиться к врачу.

```

Рисунок 10 – Тест 2.

Тест 3: Симптомы мононуклеоза

- Входные данные: высокая температура, сильная боль в горле, увеличенные лимфоузлы, усталость
- Результат: мононуклеоз (0,85)

```

Есть ли у вас симптом: температура_выше_39? (да/нет): нет
Есть ли у вас симптом: тошнота? (да/нет): нет
Есть ли у вас симптом: увеличенные_лимфоузлы? (да/нет): да
Есть ли у вас симптом: увеличенные_миндалины? (да/нет): нет
Есть ли у вас симптом: усталость? (да/нет): да
Есть ли у вас симптом: хрипы_в_груди? (да/нет): нет
Есть ли у вас симптом: цианоз? (да/нет): нет

--- Результаты диагностики ---

--- Возможные диагнозы (по убыванию уверенности) ---
1. мононуклеоз: уверенность 0.85 (85%)

--- Рекомендации ---
С высокой вероятностью у вас мононуклеоз. Рекомендуется обратиться к врачу.

```

Рисунок 11 – Тест 3.

Тест 4: Все симптомы


```

Есть ли у вас симптом: усталость? (да/нет): да
    Уверенность в наличии симптома (0-1, по умолчанию 1): 1
Есть ли у вас симптом: хрипы_в_груди? (да/нет): да
    Уверенность в наличии симптома (0-1, по умолчанию 1): 1
Есть ли у вас симптом: цианоз? (да/нет): да
    Уверенность в наличии симптома (0-1, по умолчанию 1): 1

--- Результаты диагностики ---

--- Возможные диагнозы (по убыванию уверенности) ---
1. covid19: уверенность 0.95 (95%)
2. пневмония: уверенность 0.95 (95%)
3. ветрянка: уверенность 0.95 (95%)
4. корь: уверенность 0.95 (95%)
5. менингит: уверенность 0.95 (95%)
6. герпес: уверенность 0.95 (95%)
7. грипп: уверенность 0.90 (90%)
8. ангина: уверенность 0.90 (90%)
9. кишечная_инфекция: уверенность 0.90 (90%)
10. краснуха: уверенность 0.90 (90%)
11. гепатит_A: уверенность 0.90 (90%)
12. генитальный_герпес: уверенность 0.90 (90%)
13. мононуклеоз: уверенность 0.85 (85%)
14. бронхит: уверенность 0.80 (80%)
15. вич_острая: уверенность 0.07 (7%)

--- Рекомендации ---
С высокой вероятностью у вас covid19. Рекомендуется обратиться к врачу.

```

Рисунок 12 – Тест 4.

Все тесты показали корректную работу системы. Система успешно обрабатывает ситуации, когда один симптом указывает на несколько заболеваний, и выдаёт все возможные диагнозы с соответствующими коэффициентами уверенности.

8. Итоговая структура программы

Программа состоит из следующих основных классов:

- Rule — представление одного продукционного правила
- KnowledgeBase — база знаний (хранение и управление правилами)
- InferenceEngine — механизм логического вывода
- ExpertSystemUI — пользовательский интерфейс

Интерфейс обычного пользователя:

```
--- Выберите тип пользователя ---
1. Обычный пользователь (получить диагноз)
2. Студент (диагностика с объяснением)
3. Эксперт (можно добавлять правила)
4. Инженер по знаниям (полный доступ к БЗ)
Ваш выбор: 1
Режим: Обычный пользователь

--- Главное меню ---
1. Пройти диагностику
2. Сменить тип пользователя
0. Выход

Выберите действие: █
```

Рисунок 13 – Интерфейс обычного пользователя.

Интерфейс студента:

```
--- Выберите тип пользователя ---
1. Обычный пользователь (получить диагноз)
2. Студент (диагностика с объяснением)
3. Эксперт (можно добавлять правила)
4. Инженер по знаниям (полный доступ к БЗ)
Ваш выбор: 2
Режим: Студент (объяснение хода рассуждений)

--- Главное меню ---
1. Пройти диагностику
2. Сменить тип пользователя
0. Выход

Выберите действие: █
```

Рисунок 14 – Интерфейс студента.

Интерфейс эксперта:

```
--- Выберите тип пользователя ---
1. Обычный пользователь (получить диагноз)
2. Студент (диагностика с объяснением)
3. Эксперт (можно добавлять правила)
4. Инженер по знаниям (полный доступ к БЗ)
Ваш выбор: 3
Режим: Эксперт (можно добавлять правила)

--- Главное меню ---
1. Пройти диагностику
2. Сменить тип пользователя
3. Управление базой знаний (добавить/удалить правило)
0. Выход

Выберите действие: █
```

Рисунок 15 – Интерфейс эксперта.

Интерфейс инженера:

```
--- Выберите тип пользователя ---
1. Обычный пользователь (получить диагноз)
2. Студент (диагностика с объяснением)
3. Эксперт (можно добавлять правила)
4. Инженер по знаниям (полный доступ к БЗ)
Ваш выбор: 4
Режим: Инженер по знаниям (полный доступ)

--- Главное меню ---
1. Пройти диагностику
2. Сменить тип пользователя
3. Управление базой знаний (добавить/удалить правило)
4. Просмотреть все правила
0. Выход

Выберите действие: |
```

Рисунок 16 – Интерфейс инженера.

Выводы

В результате выполнения лабораторной работы была успешно разработана экспертная система, базирующаяся на правилах, предназначенная для диагностики инфекционных заболеваний человека по совокупности симптомов. Создана база знаний, содержащая 33 продукционных правила, каждое из которых включает набор симптомов, заключение о заболевании и коэффициент уверенности от 0 до 1. Реализован механизм прямого логического вывода (forward chaining), который позволяет системе активировать все подходящие правила, вычислять итоговую уверенность каждого диагноза как произведение уверенности пользователя в симптомах и уверенности правила, а в случае множественного вывода одного заболевания выбирать максимальное значение. Также разработана рабочая память для хранения фактов и выведенных заключений, организован диалоговый интерфейс на ограниченном естественном языке, поддерживающий ветвящееся взаимодействие с пользователем.

Полностью выполнены требования по обеспечению работы пяти типов пользователей. Обычные пользователи получают быстрый диагноз без лишней информации. Студенты могут изучать ход рассуждений системы с пошаговым объяснением активированных правил. Эксперты имеют возможность добавлять новые правила в базу знаний без остановки системы. Инженеры по знаниям наделены полным набором функций: добавление, удаление и просмотр всех правил. Проведённое тестирование подтвердило корректную работу системы на различных наборах симптомов, включая случаи, когда один симптом указывает на несколько заболеваний. Все требования лабораторной работы выполнены в полном объёме, поставленная цель достигнута.

Листинг

```
# Структура правила
class Rule:
    def __init__(self, name: str, conditions: List[str], conclusion:
str, confidence: float):
        self.name = name                # Имя правила (например, "R1")
        self.conditions = conditions    # Список симптомов (условий)
        self.conclusion = conclusion    # Заключение (болезнь)
        self.confidence = confidence    # Уверенность правила (0..1)

    def __repr__(self):
        return f"{self.name}: ЕСЛИ {' '.join(self.conditions)} ТО
{self.conclusion} (уверенность: {self.confidence})"

# База знаний
class KnowledgeBase:
    def __init__(self):
        self.rules: List[Rule] = []
        self._init_rules()

    def _init_rules(self):
        # Инфекционные заболевания и их симптомы (с разной степенью
уверенности)
        # Грипп
        self.rules.append(Rule("R1", ["высокая_температура", "озноб",
"головная_боль", "ломота_в_теле"], "грипп", 0.9))
        self.rules.append(Rule("R2", ["высокая_температура", "кашель",
"боль_в_горле"], "грипп", 0.7))
        self.rules.append(Rule("R3", ["температура", "слабость",
"насморк"], "грипп", 0.6))

        # COVID-19
        self.rules.append(Rule("R4", ["высокая_температура",
"сухой_кашель", "потеря_обоняния", "одышка"], "covid19", 0.95))
        self.rules.append(Rule("R5", ["потеря_вкуса", "потеря_обоняния",
"усталость"], "covid19", 0.85))
        self.rules.append(Rule("R6", ["температура", "сухой_кашель",
"боль_в_груди"], "covid19", 0.75))

        # Ангина (стрептококковая инфекция)
        self.rules.append(Rule("R7", ["высокая_температура",
"сильная_боль_в_горле", "увеличенные_миндалины"], "ангина", 0.9))
        self.rules.append(Rule("R8", ["боль_в_горле", "гнойные_пробки",
"температура"], "ангина", 0.85))
        self.rules.append(Rule("R9", ["боль_при_глотании",
"увеличенные_лимфоузлы", "температура"], "ангина", 0.7))

        # Бронхит
        self.rules.append(Rule("R10", ["влажный_кашель", "температура",
"слабость"], "бронхит", 0.8))
        self.rules.append(Rule("R11", ["кашель_с_мокротой",
"хрипы_в_груди", "одышка"], "бронхит", 0.85))
        self.rules.append(Rule("R12", ["сухой_кашель", "боль_в_груди",
"температура"], "бронхит", 0.7))

        # Пневмония
```

```

        self.rules.append(Rule("R13", ["высокая_температура",
"кашель_с_кровью", "одышка", "боль_в_груди"], "пневмония", 0.95))
        self.rules.append(Rule("R14", ["озноб", "сильный_кашель",
"слабость", "потливость"], "пневмония", 0.85))
        self.rules.append(Rule("R15", ["температура_выше_39",
"затрудненное_дыхание", "цианоз"], "пневмония", 0.9))

    # Кишечная инфекция
        self.rules.append(Rule("R16", ["тошнота", "рвота", "диарея",
"температура"], "кишечная_инфекция", 0.9))
        self.rules.append(Rule("R17", ["боль_в_животе", "диарея",
"слабость"], "кишечная_инфекция", 0.8))
        self.rules.append(Rule("R18", ["рвота", "температура",
"обезвоживание"], "кишечная_инфекция", 0.75))

    # Ветряная оспа
        self.rules.append(Rule("R19", ["температура",
"сыпь_пузырьковая", "зуд"], "ветрянка", 0.95))
        self.rules.append(Rule("R20", ["сыпь_на_всем_теле",
"температура", "слабость"], "ветрянка", 0.85))

    # Корь
        self.rules.append(Rule("R21", ["высокая_температура", "кашель",
"сыпь", "пятна_бельского"], "корь", 0.95))
        self.rules.append(Rule("R22", ["температура", "сыпь",
"светобоязнь"], "корь", 0.85))

    # Краснуха
        self.rules.append(Rule("R23", ["температура",
"увеличенные_лимфоузлы", "розовая_сыпь"], "краснуха", 0.9))
        self.rules.append(Rule("R24", ["сыпь", "боль_в_суставах",
"температура"], "краснуха", 0.75))

    # Менингит
        self.rules.append(Rule("R25", ["сильная_головная_боль",
"ригидность_затылочных_мышц", "высокая_температура"], "менингит",
0.95))
        self.rules.append(Rule("R26", ["тошнота", "светобоязнь",
"спутанность_сознания", "температура"], "менингит", 0.9))

    # Гепатит А
        self.rules.append(Rule("R27", ["желтуха", "температура",
"тошнота", "темная_моча"], "гепатит_А", 0.9))
        self.rules.append(Rule("R28", ["боль_в_правом_подреберье",
"слабость", "потеря_аппетита"], "гепатит_А", 0.8))

    # ВИЧ-инфекция (острая стадия)
        self.rules.append(Rule("R29", ["температура",
"увеличенные_лимфоузлы", "сыпь", "боль_в_горле"], "вич_острая", 0.7))
        self.rules.append(Rule("R30", ["ночная_потливость",
"потеря_веса", "длительная_лихорадка"], "вич_острая", 0.75))

    # Герпес
        self.rules.append(Rule("R31", ["пузырьки_на_губах", "жжение",
"зуд"], "герпес", 0.95))
        self.rules.append(Rule("R32", ["пузырьки_на_гениталиях",
"температура", "увеличенные_лимфоузлы"], "генитальный_герпес", 0.9))

```

```

        # Мононуклеоз
        self.rules.append(Rule("R33", ["высокая_температура",
"сильная_боль_в_горле", "увеличенные_лимфоузлы", "усталость"],
"мононуклеоз", 0.85))

    def get_rules(self) -> List[Rule]:
        return self.rules

    def add_rule(self, name: str, conditions: List[str], conclusion:
str, confidence: float):
        self.rules.append(Rule(name, conditions, conclusion,
confidence))

    def remove_rule(self, name: str) -> bool:
        for i, r in enumerate(self.rules):
            if r.name == name:
                self.rules.pop(i)
                return True
        return False

# Механизм вывода
class InferenceEngine:
    def __init__(self, knowledge_base: KnowledgeBase):
        self.kb = knowledge_base
        self.facts: Dict[str, float] = {} # симптом -> уверенность
(0..1)
        self.derived_diseases: Dict[str, float] = {} # болезнь ->
уверенность
        self.explanation: List[str] = [] # для режима студента

    def add_fact(self, symptom: str, confidence: float = 1.0):
        self.facts[symptom] = confidence

    def reset(self):
        self.facts = {}
        self.derived_diseases = {}
        self.explanation = []

    def forward_chaining(self, explain_mode: bool = False) -> Dict[str,
float]:
        """Прямой цепочка рассуждений: выводит все возможные болезни"""
        changed = True
        while changed:
            changed = False
            for rule in self.kb.get_rules():
                # Проверяем, не выведено ли уже это заключение
                if rule.conclusion in self.derived_diseases:
                    continue

                # Проверяем, все ли условия выполняются
                rule_confidence = 1.0
                all_conditions_met = True

                for cond in rule.conditions:
                    if cond in self.facts:
                        rule_confidence *= self.facts[cond]

```

```

        else:
            all_conditions_met = False
            break

        if all_conditions_met:
            # Уверенность правила умножается на уверенность
условий
            final_conf = rule_confidence * rule.confidence
            if rule.conclusion in self.derived_diseases:
                self.derived_diseases[rule.conclusion] = max(
                    self.derived_diseases[rule.conclusion],
final_conf
                )
            else:
                self.derived_diseases[rule.conclusion] =
final_conf

            changed = True

            if explain_mode:
                self.explanation.append(
                    f"Активировано правило {rule.name}: {'',
'.join(rule.conditions)} → {rule.conclusion} (уверенность:
{final_conf:.2f})"
                )

        sorted_diseases = dict(sorted(self.derived_diseases.items(),
key=lambda x: x[1], reverse=True))
        return sorted_diseases

    def get_all_symptoms(self) -> List[str]:
        """Получить все возможные симптомы из базы знаний"""
        symptoms = set()
        for rule in self.kb.get_rules():
            for cond in rule.conditions:
                symptoms.add(cond)
        return sorted(symptoms)

# Пользовательский интерфейс
class ExpertSystemUI:
    def __init__(self):
        self.kb = KnowledgeBase()
        self.engine = InferenceEngine(self.kb)
        self.current_user_type = "user"

    def run(self):
        print("\n" + "="*60)
        print("ЭКСПЕРТНАЯ СИСТЕМА: Диагностика инфекционных
заболеваний")
        print("="*60)

        self.choose_user_type()

        while True:
            print("\n--- Главное меню ---")
            print("1. Пройти диагностику")
            print("2. Сменить тип пользователя")

```

```

        if self.current_user_type in ["expert",
"knowledge_engineer"]:  

            print("3. Управление базой знаний (добавить/удалить  

правило)")  

            if self.current_user_type == "knowledge_engineer":  

                print("4. Просмотреть все правила")  

                print("0. Выход")  

  

                choice = input("\nВыберите действие: ")  

  

                if choice == "1":  

                    self.run_diagnosis()  

                elif choice == "2":  

                    self.choose_user_type()  

                elif choice == "3" and self.current_user_type in ["expert",  

"knowledge_engineer"]:  

                    self.manage_knowledge_base()  

                    elif choice == "4" and self.current_user_type ==  

"knowledge_engineer":  

                        self.show_all_rules()  

                elif choice == "0":  

                    print("До свидания!")  

                    break  

                else:  

                    print("Неверный ввод. Попробуйте снова.")  

  

def choose_user_type(self):  

    print("\n--- Выберите тип пользователя ---")  

    print("1. Обычный пользователь (получить диагноз)")  

    print("2. Студент (диагностика с объяснением)")  

    print("3. Эксперт (можно добавлять правила)")  

    print("4. Инженер по знаниям (полный доступ к БЗ)")  

  

    choice = input("Ваш выбор: ")  

    if choice == "1":  

        self.current_user_type = "user"  

        print("Режим: Обычный пользователь")  

    elif choice == "2":  

        self.current_user_type = "student"  

        print("Режим: Студент (объяснение хода рассуждений)")  

    elif choice == "3":  

        self.current_user_type = "expert"  

        print("Режим: Эксперт (можно добавлять правила)")  

    elif choice == "4":  

        self.current_user_type = "knowledge_engineer"  

        print("Режим: Инженер по знаниям (полный доступ)")  

    else:  

        print("Неверный выбор. Установлен режим обычного  

пользователя.")  

        self.current_user_type = "user"  

  

def run_diagnosis(self):  

    self.engine.reset()  

  

    print("\n--- Диагностика инфекционных заболеваний ---")  

    print("Отвечайте 'да' или 'нет' на вопросы о симптомах.\n")

```



```

all_symptoms = self.engine.get_all_symptoms()

for symptom in all_symptoms:
    answer = input(f"Есть ли у вас симптом: {symptom}? (да/нет): ")
    answer = answer.lower()
    if answer == "да":
        if self.current_user_type in ["expert", "knowledge_engineer"]:
            conf_input = input(f" Уверенность в наличии симптома (0-1, по умолчанию 1): ")
            try:
                conf = float(conf_input)
                conf = max(0, min(1, conf))
            except:
                conf = 1.0
        else:
            conf = 1.0
        self.engine.add_fact(symptom, conf)

if not self.engine.facts:
    print("\nСимптомы не введены. Невозможно поставить диагноз.")
    return

print("\n--- Результаты диагностики ---")

explain = (self.current_user_type == "student")
diseases = self.engine.forward_chaining(explain_mode=explain)

if explain and self.engine.explanation:
    print("\n--- Ход рассуждений ---")
    for step in self.engine.explanation:
        print(f" • {step}")

if not diseases:
    print("Не удалось определить заболевание по указанным симптомам.")
    print("Рекомендуется обратиться к врачу для дополнительного обследования.")
else:
    print("\n--- Возможные диагнозы (по убыванию уверенности) ---")
    for i, (disease, conf) in enumerate(diseases.items(), 1):
        print(f"{i}. {disease}: уверенность {conf:.2f} ({conf*100:.0f}%)")

    # Рекомендации
    print("\n--- Рекомендации ---")
    top_disease = list(diseases.keys())[0]
    if diseases[top_disease] > 0.8:
        print(f"С высокой вероятностью у вас {top_disease}. Рекомендуется обратиться к врачу.")
    elif diseases[top_disease] > 0.5:
        print(f"Возможно, у вас {top_disease}. Следите за симптомами и при ухудшении обратитесь к врачу.")
    else:
        print("Симптомы неоднозначны. Рекомендуется консультация специалиста.")

```

```

def manage_knowledge_base(self):
    print("\n--- Управление базой знаний ---")
    print("1. Добавить новое правило")
    print("2. Удалить правило")
    print("3. Показать все правила")
    print("0. Назад")

    choice = input("Выбор: ")
    if choice == "1":
        self.add_new_rule()
    elif choice == "2":
        self.remove_rule()
    elif choice == "3":
        self.show_all_rules()
    elif choice == "0":
        return
    else:
        print("Неверный выбор.")

def add_new_rule(self):
    print("\n--- Добавление нового правила ---")
    name = input("Имя правила (например, R34): ").strip()

    print("Введите симптомы (условия) через запятую (например: температура, кашель, боль):")
    cond_input = input("Симптомы: ").strip()
    conditions = [c.strip() for c in cond_input.split(",")]

    conclusion = input("Заключение (название болезни): ").strip()

    conf_input = input("Уверенность правила (0-1): ").strip()
    try:
        confidence = float(conf_input)
        confidence = max(0, min(1, confidence))
    except:
        confidence = 0.5

    self.kb.add_rule(name, conditions, conclusion, confidence)
    print(f"Правило {name} успешно добавлено!")

def remove_rule(self):
    self.show_all_rules()
    name = input("\nВведите имя правила для удаления: ").strip()
    if self.kb.remove_rule(name):
        print(f"Правило {name} удалено.")
    else:
        print(f"Правило {name} не найдено.")

def show_all_rules(self):
    print("\n--- Все правила базы знаний ---")
    for i, rule in enumerate(self.kb.get_rules(), 1):
        print(f"{i}. {rule}")

if __name__ == "__main__":
    ui = ExpertSystemUI()
    ui.run()

```